Experiment 07: Deploying and Managing Containerized Applications using Kubernetes

Experiment No.	07
Mapped CO	CO4
Aim	To deploy and manage containerized applications using Kubernetes.
Tools / Software	Minikube or kind local cluster; kubectl; Docker Engine; A container image from earlier Docker experiment

Theory

Kubernetes is a container orchestration platform used to deploy, expose, scale and manage containerized applications across a cluster. It automates many operational tasks such as scheduling containers, maintaining the desired number of running instances and restarting failed workloads.

The smallest deployable unit in Kubernetes is a Pod. A Deployment manages Pods through ReplicaSets and keeps the desired number of replicas running. A Service provides a stable network endpoint for accessing the Pods, even when Pods are recreated or scaled.

Kubernetes follows a declarative model. The user writes YAML manifest files describing the desired state, such as number of replicas, container image, exposed port and service type. Kubernetes controllers continuously compare actual state with desired state and reconcile the cluster accordingly. This makes applications more reliable, scalable and manageable.

Procedure

1. Install Docker, kubectl and Minikube or kind on the local machine.
2. Start the Kubernetes cluster using minikube start or kind create cluster.
3. Verify the cluster using kubectl get nodes.
4. Prepare or pull a container image created in the Docker experiment, for example <user>/myapp:1.0.
5. Create a deployment.yaml file specifying application name, container image, container port and replica count.
6. Create a service.yaml file of type NodePort to expose the application outside the cluster.
7. Apply both YAML files using kubectl apply -f deployment.yaml -f service.yaml.
8. Check Pods, Deployments and Services using kubectl get pods, kubectl get deployments and kubectl get svc.
9. Access the application using minikube service myapp-svc --url or through the NodePort URL.
10. Scale the deployment using kubectl scale deployment myapp --replicas=4 and observe the new Pods.
11. Delete one Pod manually and verify Kubernetes self-healing by checking that a replacement Pod is created.
12. Clean up resources using kubectl delete -f deployment.yaml -f service.yaml and stop the local cluster.

Sample Code / Commands

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
```

```
    spec:
      containers:
      - name: myapp
        image: <user>/myapp:1.0
        ports:
        - containerPort: 5000
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-svc
spec:
  type: NodePort
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 5000
```

Kubernetes commands

```
minikube start
kubectl get nodes
kubectl apply -f deployment.yaml -f service.yaml
kubectl get pods
kubectl get deployments
kubectl get svc
minikube service myapp-svc --url
kubectl scale deployment myapp --replicas=4
kubectl get pods
kubectl delete pod <pod-name>
kubectl get pods
kubectl delete -f deployment.yaml -f service.yaml
minikube stop
```

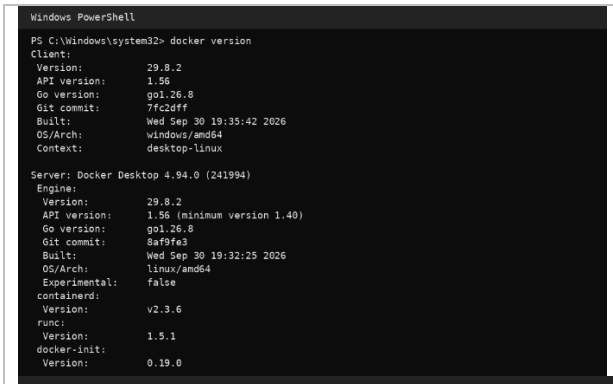
Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Kubernetes cluster setup, a pplication deployment, service exposure, scaling and self-healing verification. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Sagar Pradhan	49	B2	02-10-26

1. Kubernetes Cluster Setup Output

Paste screenshot/output showing minikube/kind start command and kubectl get nodes output.




```
Windows PowerShell

PS C:\Windows\system32> kubectl get nodes
NAME             STATUS   ROLES           AGE    VERSION
docker-desktop   Ready    control-plane   105s   v1.36.1

PS C:\Windows\system32> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:6443
CoreDNS is running at https://127.0.0.1:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

2. Deployment Manifest Output

Paste screenshot/output showing deployment.yaml with image name, replicas, labels and container port.

```
Windows PowerShell

PS C:\Users\hp5cd\docker-flask-demo> Get-Content deployment.yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mywebapp-deployment
  labels:
    app: mywebapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: mywebapp
  template:
    metadata:
      labels:
        app: mywebapp
    spec:
      containers:
        - name: mywebapp
          image: mywebapp:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
```

```
Windows PowerShell

PS C:\Windows\system32> docker images
IMAGE                                                                                                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/desktop-kubernetes:kubernetes-v1.36.1-cni-v1.9.1-critools-v1.35.0-cri-dockerd-v0.4.4-1-debian
                                                                                                     65e5eddaacf5        594MB          188MB
docker/desktop-storage-provisioner:v4.0                                                              ca46f2a97ef4       82.1MB         24.7MB    U
docker/desktop-vpnkit-controller:v4.0                                                                8cb4e79ec634       52.6MB         11.9MB    U
myapp:1.0                                                                                            d8ab5770f3b8        198MB         48.3MB
mywebapp:1.0                                                                                         d8ab5770f3b8        198MB         48.3MB
registry.k8s.io/coredns/coredns:v1.14.2                                                              e7e6440cfd1e        103MB         23.3MB    U
registry.k8s.io/etcd:3.6.8-0                                                                         397189418d1a       88.8MB         22.9MB    U
registry.k8s.io/kube-apiserver:v1.36.1                                                               2460df74307c        128MB           29MB    U
registry.k8s.io/kube-controller-manager:v1.36.1                                                      c8adb8daee5b        114MB         25.7MB    U
registry.k8s.io/kube-proxy:v1.36.1                                                                   a96b6e12863e        111MB         28.9MB    U
registry.k8s.io/kube-scheduler:v1.36.1                                                               3a4333e8ba6e       82.7MB         19.7MB    U
registry.k8s.io/pause:3.10.2                                                                         f548e0e8e3dc       1.06MB          318kB    U
```

3. Service Manifest Output

Paste screenshot/output showing service.yaml with NodePort/LoadBalancer type, selector and target port.

```
Windows PowerShell

PS C:\Users\hp5cd\docker-flask-demo> Get-Content service.yaml

apiVersion: v1
kind: Service
metadata:
  name: mywebapp-service
spec:
  type: NodePort
  selector:
    app: mywebapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
      nodePort: 30080
```

```
Windows PowerShell

PS C:\Users\hp5cd\docker-flask-demo> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
mywebapp-deployment   2/2     2            2           106s

PS C:\Users\hp5cd\docker-flask-demo> kubectl get pods -o wide
NAME                                 READY   STATUS    RESTARTS   AGE    IP         NODE             NOMINATED NODE   READINESS GATES
mywebapp-deployment-f64cd58c6-kbblp   1/1     Running   0          2m7s   10.1.0.6   docker-desktop   <none>           <none>
mywebapp-deployment-f64cd58c6-t5jcb   1/1     Running   0          2m7s   10.1.0.7   docker-desktop   <none>           <none>
```

4. Apply and Resource Status Output

Paste screenshot/output showing kubectl apply and kubectl get pods, deployments and services output.

```
Windows PowerShell

PS C:\Users\hp5cd\docker-flask-demo> kubectl logs mywebapp-deployment-f64cd58c6-hxbpk
 * Serving Flask app 'app'
 * Debug mode: off
 WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.1.0.8:5000
 Press CTRL+C to quit

PS C:\Users\hp5cd\docker-flask-demo> kubectl describe pod mywebapp-deployment-f64cd58c6-hxbpk
Name:             mywebapp-deployment-f64cd58c6-hxbpk
Namespace:        default
Node:             docker-desktop/192.168.65.3
Status:           Running
IP:               10.1.0.8
Controlled By:    ReplicaSet/mywebapp-deployment-f64cd58c6
Containers:
  mywebapp:
    Image:          mywebapp:1.0
    Port:           5000/TCP
    State:          Running
    Ready:          True
    Restart Count:  0
Events:
  Normal  Scheduled  Successfully assigned to docker-desktop
  Normal  Pulled     Container image already present
  Normal  Created    Container created
  Normal  Started    Container started
```

```
Windows PowerShell

PS C:\Users\hp5cd\docker-flask-demo> kubectl scale deployment mywebapp-deployment --replicas=4
deployment.apps/mywebapp-deployment scaled
PS C:\Users\hp5cd\docker-flask-demo> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
mywebapp-deployment   4/4     4            4           7m30s

PS C:\Users\hp5cd\docker-flask-demo> kubectl get pods -o wide
NAME                                 READY   STATUS    RESTARTS   AGE     IP         NODE             NOMINATED NODE   READINESS GATES
mywebapp-deployment-f64cd58c6-hxbpk   1/1     Running   0          50s     10.1.0.8   docker-desktop   <none>           <none>
mywebapp-deployment-f64cd58c6-kbblp   1/1     Running   0          7m54s   10.1.0.6   docker-desktop   <none>           <none>
mywebapp-deployment-f64cd58c6-p9lh7   1/1     Running   0          50s     10.1.0.9   docker-desktop   <none>           <none>
mywebapp-deployment-f64cd58c6-t5jcb   1/1     Running   0          7m54s   10.1.0.7   docker-desktop   <none>           <none>
```

5. Application Access Output

Paste screenshot/output showing minikube service URL, browser output or curl response.

```
Windows PowerShell

PS C:\Users\hp5cd\docker-flask-demo> curl.exe http://localhost:30080
Hello from Docker container! Application deployed successfully.
```

6. Scaling and Self-Healing Output

Paste screenshot/output showing kubectl scale command, increased Pod count and replacement Pod after deletion.

```
Windows PowerShell

PS C:\Users\hp5cd\docker-flask-demo> kubectl scale deployment mywebapp-deployment --replicas=4
deployment.apps/mywebapp-deployment scaled
PS C:\Users\hp5cd\docker-flask-demo> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
mywebapp-deployment   4/4     4            4           7m30s

PS C:\Users\hp5cd\docker-flask-demo> kubectl get pods -o wide
NAME                                 READY   STATUS    RESTARTS   AGE     IP         NODE             NOMINATED NODE   READINESS GATES
mywebapp-deployment-f64cd58c6-hxbpk   1/1     Running   0          50s     10.1.0.8   docker-desktop   <none>           <none>
mywebapp-deployment-f64cd58c6-kbblp   1/1     Running   0          7m54s   10.1.0.6   docker-desktop   <none>           <none>
mywebapp-deployment-f64cd58c6-p9lh7   1/1     Running   0          50s     10.1.0.9   docker-desktop   <none>           <none>
mywebapp-deployment-f64cd58c6-t5jcb   1/1     Running   0          7m54s   10.1.0.7   docker-desktop   <none>           <none>
```

GitHub Repository / Lab Folder Link:

_____

Docker Hub / Container Image Link:

_____

Final Observation Statement:

```
Write 2-3 lines summarizing the result of the experiment.
```

Expected Output

A local Kubernetes cluster should start successfully. The Deployment should create the requested number of Pods. The Service should expose the application through a stable endpoint. The application should be reachable through the Service URL. Scaling should increase the number of Pods, and self-healing should recreate a Pod if one is deleted.

Conclusion

A containerized application was deployed and managed on Kubernetes. The experiment demonstrated declarative deployment using YAML manifests, service exposure, scaling of replicas and Kubernetes self-healing behavior.

Viva / Review Questions

1. What is a Pod and how does it differ from a container?
2. What is the role of a ReplicaSet within a Deployment?
3. Differentiate ClusterIP, NodePort and LoadBalancer Service types.
4. How does Kubernetes achieve self-healing?

Marks: ____ / 10 **Date: _____** **Signature of Faculty: _____**